

32 张 NPU、6 或 7 个 SSU：怎样判断 I/O stall？

逐盘排队、汇聚链路、严格下界与在线条件预测

2026-09-06 · 配套多 SSU Python 实现

目录

1 先说结论：能够扩展，但不能只把 40 改成 240 或 280	2
2 先认识单位和时间点	3
2.1 一条 I/O 到底多大	3
2.2 一层的五个时间点	3
3 通俗理解：多个仓库，一个卸货口	4
3.1 一个从 6 盘变 7 盘完全不改善的例子	4
4 当前层：只有计数，也能判定一部分必然失败的情况	5
4.1 这个接口必须满足什么条件	5
4.2 第一步：每盘最乐观还要忙多久	5
4.3 第二步：检查整层数据能否通过一个接收口	6
4.4 第三步：只给有依据的判断	7
4.5 最小调用：当前层是否必然来不及	7
4.6 已经提交了一部分：改用目标自己的阶段计数	7
5 要看后续层，必须把“计算—预取—排队”闭环推下去	8
5.1 每个盘的 FIFO 递推	8
5.2 每张卡的接收 FIFO 递推	9
5.3 从读取完成，回到计算和下一次预取	9
6 怎样证明：某种长突发连自然错开也躲不过	9
7 客户端究竟要提供多少信息	10
7.1 三个信息等级，对应三种不同承诺	10
7.2 NPUSate 最容易填错的地方	11
7.3 同样的总队长为什么会有不同结果	11
8 当前请求、新请求：最小可用的 Python 调用	11

1 先说结论：能够扩展，但不能只把 40 改成 240 或 280	2
8.1 用完整已知请求做冷启动闭环预测	11
8.2 新请求进入后，分别检查自己和别人	12
9 哪些等待有机会优化，哪些不能靠 QoS 消除	13
9.1 先算每盘和每卡的长期需求	13
9.2 单层独占仍来不及，不该怪其他 NPU	14
9.3 容量检查通过，也不等于无 stall	14
10 策略 A：分盘、分卡联合规划 CIR	14
10.1 不只是给每个 SSU 各做一次独立 EDF	14
10.2 已实现的可比较规则	15
10.3 CIR 不是实际瞬时速率，也不是 PIR	15
11 策略 B：新请求到达时再选 NPU	15
12 怎样判断“数学和 Python 匹配仿真”	16
12.1 需要分别报告冷态和暖态	16
12.2 本轮已经验证到了什么范围	17
12.3 策略比较还要防止三个偏差	17
13 最后整理成实际使用顺序	18

1 先说结论：能够扩展，但不能只把 40 改成 240 或 280

这里的 32 张 NPU 各自处理不同请求，不是 32 张卡共同计算一个请求。每张卡一时只计算一个请求，每个请求 8 层，允许预取下一层，也开启已有下一请求的跨请求预取。KV 数据走 SSD→HBM，不经过 DRAM。

从一个 SSU 扩展到多个 SSU，至少增加了两个必须计算的因素：

1. **每盘分别排队。**总带宽没有用满，不代表每盘都有余量。某层只要有一个 SSU 的部分数据迟到，这层就不能开始计算。
2. **多盘共用每张卡的一个接收口。**6 个盘各有 40 GiB/s，不代表一张卡可以接收 240 GiB/s；本项目每卡接收总上限仍然是 50 GiB/s。

因此，原来的“前方工作时间+自己的读取时间”和多层反馈思想仍适用，但必须变成**每个 SSU 的 FIFO，加每张 NPU 的接收 FIFO**。只有各盘总排队数，通常不能精确预测未来各层。

配套实现分工如下：

- multi_ssu_stall_predictor.py：Baseline 的严格单层下界、独立多层预测、新请求影响比较。
- multi_ssu_qos_controller.py：策略 A，固定 NPU 分配，动态选定专用 Path 的 CIR。
- multi_ssu_npu_assignment.py：策略 B，在 A 之上，于实际到达时选择 NPU。

这三个结论不能混为一谈：公式下界成立，不等于在线预测一定精确；在线预测精确，也不等于某个调度策略一定提升利用率。性能数字和验证误差应以配套实验报告及原始 JSON 为准。

2 先认识单位和时间点

2.1 一条 I/O 到底多大

本说明固定：

$$b = 176 \text{ KiB} = 180224 \text{ byte.}$$

这是 GLM 格式下 128 token 的一层 KV 数据，**一条 I/O 就是一个完整 KV block**。不是 128 个这种 KV block，也不是 4 KiB。

用 GiB/s 输入带宽时，设 B_s 是 SSU s 的带宽， L_i 是 NPU i 的接收带宽。一条 I/O 的两个服务时间分别为：

$$\tau_s = \frac{b}{B_s 2^{30}} 1000 \text{ ms}, \quad \ell_i = \frac{b}{L_i 2^{30}} 1000 \text{ ms}.$$

本项目使用所有盘 $B_s = 40$ 、所有卡 $L_i = 50$ ，因此：

$$\tau = 0.0041961669921875 \text{ ms},$$

$$\ell = 0.00335693359375 \text{ ms}.$$

Python 多盘预测器目前接受统一的 `disk_gib_s` 和 `link_gib_s`；下面有下标的数学记法也便于说明异构设备，但不能据此误认为所有现有接口都已支持逐盘不同带宽。

2.2 一层的五个时间点

符号	通俗含义
$U_{i,l}$	第 l 层读取被激活，客户端可以开始逐条发 I/O
$R_{i,l}$	第 l 层的全部数据已经进入 NPU 的 HBM
$S_{i,l}$	第 l 层实际开始计算
$E_{i,l}$	第 l 层计算结束
$D_{i,l}$	希望第 l 层的数据最迟到齐的时间

对预取的下一层，项目使用：

$$U_{i,l+1} = S_{i,l}, \quad D_{i,l+1} = E_{i,l} = S_{i,l} + C_i.$$

其中 C_i 是当前请求每层计算时长。各 NPU 请求不同， C_i 也可以不同。即使 C_i 固定，在当前时间 t 剩下的预算仍是 $D_{i,l+1} - t$ ，并非一直等于 C_i 。

下一层实际等待：

$$W_{i,l+1} = \max(0, R_{i,l+1} - E_{i,l}).$$

项目中的 request_id 只是全局唯一的请求编号。例如 1001 表示“编号 1001 的那个请求”，不是长度、层数或队列位置。layer=0 表示它的第一层。日志必须同时看 request_id 和 layer，不能把不同请求的 L0 混成同一层。

跨请求预取还有一个限制：只有当前请求最后一层**开始计算时已经到达**的下一请求，才会被预取。之后才到达的请求不能假装在过去已经读了数据。

3 通俗理解：多个仓库，一个卸货口

把每个 SSU 想成一个独立仓库；每个 NPU 的 HBM 接收链路是这个客户自己的卸货口。

一个客户要开始做下一道菜，必须收齐来自各仓库的全部材料。这里有两种完全不同的等待：

- 仓库 A 的货车队很长。仓库 B、C 的材料已送到，也不能替代缺少的 A 的材料。
- 各仓库同时送来很多货，但客户只有一个卸货口。即使所有仓库都没有别人排队，卸货本身也需要时间。

自然错开只是在改变“什么时候加入哪些队列”。它不能增加仓库吞吐，也不能把一个卸货口变成六个。

3.1 一个从 6 盘变 7 盘完全不改善的例子

取原始 data 的 (192K, nql=128)：

- 每层要读 1535 个完整 KV block。
- 数据量为 $1535b = 0.2576446533203125$ GiB。
- 每层计算 $C = 4.533469108194381$ ms。

暂时只有一张卡发请求，其余卡不发请求。把数据散到 6 个或 7 个盘后，**仅接收这些数据**就至少需要：

$$\frac{0.2576446533203125}{50} 1000 = 5.15289306640625 \text{ ms.}$$

加上第一条 SSD 命令的启动服务时间，仿真中这个例子每层读取为：

$$R - U = \tau + 1535\ell = 5.1570892333984375 \text{ ms.}$$

因为接收口从第一条数据到达后一直忙，后面的数据持续排队，6 盘和 7 盘结果相同。L1-L7 每层的 stall 都是：

$$5.1570892333984375 - 4.533469108194381 = 0.623620125204056 \text{ ms.}$$

这是独立 Python 和原仿真器都能复现的**链路机制反例**，不是 32 张卡同时繁忙时的平均利用率结论。它说明不能预先保证“多一个盘就能去掉 stall”。

4 当前层：只有计数，也能判定一部分必然失败的情况

4.1 这个接口必须满足什么条件

screen_current_layer 针对的是：**目标这一层的所有 I/O 都尚未提交，而且全部数据在各 SSU 上都使用严格 FIFO 的 Path0。**

对 NPU i 的这个目标层，输入：

输入	口径
K_s	目标层在 SSU s 上要读多少条 I/O
Q_s	该盘 Path0 未完成的 I/O 数，包含正在服务的一条，不含目标
r_s	该盘正在服务的 I/O 还剩多少 byte；可未知
Q^{link}	该 NPU 接收队列已有的 I/O 数，包含正在接收的一条
r^{link}	正在接收的一条还剩多少 byte；可未知
D	从现在起还能等多久，ms

这时不需要知道旧队列里哪个 NPU 排在谁前面：目标还没提交，旧队列里的全部工作都在目标前方。

如果目标已经有一部分 I/O 提交了， Q_s 可能包含自己的数据和排在自己后面的数据。**不要再把整个 Q_s 当成前方工作量。**应改用后面的部分状态预测，或者补充足够的信息计算专门的界限。

4.2 第一步：每盘最乐观还要忙多久

当在途剩余量已知时：

$$A_s = \begin{cases} 0, & Q_s = 0, \\ (Q_s - 1)\tau_s + (r_s/b)\tau_s, & Q_s > 0. \end{cases}$$

若 r_s 未知，为了构造**下界**，把它当成 0：

$$A_s^{LB} = \max(Q_s - 1, 0)\tau_s.$$

这是故意乐观：不是说真实在途 I/O 没有工作，而是假设它马上结束。

对 $K_s > 0$ 的盘，目标第一条、最后一条离开 SSD 的时刻分别不可能早于：

$$F_s^{first} = A_s^{LB} + \tau_s, \quad F_s^{last} = A_s^{LB} + K_s\tau_s.$$

因此所有盘的数据进入 HBM 至少需要：

$$T_{disk}^{LB} = \max_{s:K_s>0} F_s^{last} + \ell_i.$$

只取最后一条的接收时间，是一个乐观下界，**不是完整的多盘接收时间公式**。

4.3 第二步：检查整层数据能否通过一个接收口

接收口已有工作的乐观剩余时间为：

$$A_{link}^{LB} = \max(Q^{link} - 1, 0)\ell_i + \mathbf{1}_{r^{link}\text{已知}}(r^{link}/b)\ell_i,$$

其中额外剩余量项仅在 $Q^{link} > 0$ 时使用。没有旧数据时取 0。

目标最早第一条数据到达接收口，也要等到：

$$F^{first} = \min_{s:K_s>0} F_s^{first}.$$

即使随后目标的全部 I/O 连续卸货、没有任何别人插入，整层最早也要：

$$T_{link}^{LB} = \max(A_{link}^{LB}, F^{first}) + \left(\sum_s K_s\right)\ell_i.$$

两个限制要同时满足，所以最后取：

$$T^{LB} = \max(T_{disk}^{LB}, T_{link}^{LB}).$$

当目标完全不需要 I/O 时，接口返回 $T^{LB} = 0$ ；它不必等旧队列。

4.4 第三步：只给有依据的判断

$$T^{LB} > D \implies \text{必然 stall, 且至少 } T^{LB} - D.$$

反过来不成立。 $T^{LB} \leq D$ 只是“最理想时来得及”，不代表实际能赶上。当前函数只返回：

- `must_stall`：严格乐观下界已经超过预算。
- `unknown`：目前无法证明一定失败，也没有证明一定成功。

`ready_upper_ms=None` 表示没有保证的有限上界。之后别的 NPU 可以继续逐条发入新 I/O；本层尚未发完的部分会受它们影响。该接口不把“以后没人插入”偷当成保证。

4.5 最小调用：当前层是否必然来不及

```
from multi_ssu_stall_predictor import screen_current_layer
```

```
check = screen_current_layer(
    kv_blocks_by_ssu=[256, 256, 256, 256, 256, 255],
    deadline_ms=4.533469108194381,
    path_outstanding_io=[0, 0, 0, 0, 0, 0],
    disk_gib_s=40,
    link_gib_s=50,
)
print(check["verdict"])           # must_stall
print(check["ready_lower_ms"])    # 5.1570892333984375
print(check["stall_lower_ms"])    # 约 0.6236201252 ms
```

换成 7 个 SSU，必须相应提供 7 个 K_s 和 7 个 Q_s ，而不是只把带宽乘 7。这个例子即使 Q_s 全为 0，也会因接收口而失败。

4.6 已经提交了一部分：改用目标自己的阶段计数

新增接口 `screen_pending_layer` 可以检查正在读取的当前层。它不再接收整个 Path 的队长，而是接收**这一层自己的**三个部分：各盘已提交未完成 q_s 、各盘尚未提交 u_s 、接收口里的目标层 I/O 数 q^{link} 。

不知道其他请求位于自己前面还是后面时，把它们全部忽略；如果自己在某盘已有 I/O，就乐观假定其中一条正在服务、马上完成。因此这个盘上的自己剩余工作最早完成时间为：

$$f_s^{last} = (q_s + u_s - \mathbf{1}_{q_s > 0})\tau_s,$$

仅对 $q_s + u_s > 0$ 的盘使用。自己第一条出盘时间的下界为 $f_s^{first} = 0$ ($q_s > 0$) 或 τ_s ($q_s = 0$)。

令 H_{link}^{own} 是本层在接收口已排队工作的乐观剩余时间。与前面相同：未知在途剩余量时允许一条马上结束，已知时加入这条**目标层 I/O 自己的**剩余量。得到：

$$T_{pending}^{LB} = \max \left(\max_s f_s^{last} + \ell_i, \max(H_{link}^{own}, \min_s f_s^{first}) + \sum_s (q_s + u_s) \ell_i \right).$$

没有 SSD 侧剩余工作时，只剩本层接收队列的下界；全部阶段计数均为 0 时，函数返回 `already_ready`，未来 stall 下界为 0，应从待检查列表中移除。它不凭空恢复已经过去的真实到齐时刻，因此也不能仅因输入 deadline 为负数就断言历史上曾错过 deadline。不要把“早已到齐的一层”当成现在才开始计时的新读取。

```
from multi_ssu_stall_predictor import screen_pending_layer
```

```
pending = screen_pending_layer(
    queued_io_by_ssu=[2, 1, 0, 0, 0, 0],
    unissued_by_ssu=[1, 1, 1, 0, 0, 0],
    deadline_ms=0.01,
    link_pending_io=1,
)
print(pending["verdict"])
print(pending["stall_lower_ms"])
```

这里有 3 条仍在 SSD、3 条还未提交、1 条在本卡接收口。无论其他人的 FIFO 顺序如何，剩余 6 条 SSD 侧 I/O 都还需要经过接收口；因此能得到保守的必然失败判断。这个下界通常比拿到真实队列顺序的预测更松，代价是漏掉一些实际会 stall 的情况，而不是凭空保证成功。

5 要看后续层，必须把“计算—预取—排队”闭环推下去

5.1 每个盘的 FIFO 递推

先假定每盘 I/O 的服务顺序已经确定。设 $a_{s,j}$ 是排在该盘第 j 个服务位置的 I/O 的提交时刻， $F_{s,j}$ 是它离开 SSD 的时刻。完整等大小 I/O 的递推为：

$$F_{s,j} = \max(a_{s,j}, F_{s,j-1}) + \tau_s.$$

含义是：“等这条命令到达，也等前一条服务完，再用一个服务时间。”每个盘都要独立维护这个递推，不能合成一个总盘队列。

正在服务的第一条若只剩 r_s byte，第一次加的是 $(r_s/b)\tau_s$ ，而不是完整 τ_s 。

5.2 每张卡的接收 FIFO 递推

把所有盘发给 NPU i 的已完成 I/O，按其到达接收口的顺序合并。设第 k 条到达时刻为 $f_{i,k}$ ，进入 HBM 的完成时刻为 $G_{i,k}$ ：

$$G_{i,k} = \max(f_{i,k}, G_{i,k-1}) + \ell_i.$$

此处的前一条可能来自另一个 SSU，甚至来自另一个已知请求。因此必须使用**每张 NPU 的总接收队列**。

同一时刻多盘完成时，也要明确稳定的排序规则。配套实现对齐了本项目的请求、层、block、SSU 同时完成排序；现实设备若排序不同，预测也应使用相应约定。

一层的数据到齐时刻是该层所有 I/O 的 HBM 完成时刻的最大值：

$$R_{i,l} = \max_{k \in \text{这一层}} G_{i,k}.$$

5.3 从读取完成，回到计算和下一次预取

$$S_{i,l} = \max(E_{i,l-1}, R_{i,l}), \quad E_{i,l} = S_{i,l} + C_i.$$

新的 $S_{i,l}$ 又会决定下一层读取的激活时刻 $U_{i,l+1}$ 。这就是自然错开的反馈来源：一次等待推迟计算，也推迟后面的读取。

但反馈不一定把冲突消除。推迟的读取可能碰上另一个长队，也可能让多个短任务在一批旧数据放行后再次相近地发起下一轮。是否最终错开，取决于全部卡的计算预算、每盘数据量、发射过程、FIFO 状态和后续请求变化，不能只看第一层是否撞车。

6 怎样证明：某种长突发连自然错开也躲不过

这里给出一个**充分条件**，不是所有低利用率输入的完整分类。

在时刻 a ，某个长请求已经把一批 I/O 全部提交。盘 s 至少还有 H_s ms 的已承诺 FIFO 工作。受影响的短计算卡当前正在计算，每层计算 C ms，每层还需要从这个盘读 K_s 条 I/O。

假设这张卡接下来两次切换都不 stall。当前计算最多再过 C ms 就结束，因此下一次计算最迟在 $a + C$ 开始；那时发起再下一层读取，它的 deadline 最迟是 $a + 2C$ 。

但是这次新发的读取不能越过已排好的 FIFO 工作，数据到齐不早于：

$$a + H_s + K_s \tau_s + \ell_i.$$

因此，只要受影响卡确实使用的某个盘满足：

$$H_s + K_s \tau_s + \ell_i > 2C,$$

就能否定“接下来两次都没有 stall”。无需提前知道这张卡正处于计算周期的哪个位置。

如果长批在 J ms 内向各盘共提交了 $K_{burst,s}$ 条 I/O，因为一个盘在此期间最多服务 J ms 的工作量，可以使用：

$$H_s^{LB} = \max(0, K_{burst,s} \tau_s - J).$$

配套函数是 `fifo_burst_certificate`。它返回的超出量是**随后两次计算切换的累计 stall 下界**，不是保证某一层单独 stall 这么久。第一次迟到会移动后面的 deadline，不能重复扣这笔时间。

使用证明需要同时满足：

- 每盘使用严格 FIFO Path0，没有后来请求越过这批工作。
- 受影响 NPU 当时正在计算，且至少还有两次后续计算切换可考察。
- 这两次切换涉及的读量和中间计算时间符合假定的固定 profile。
- 只有一层前瞻预取，不能提前多层把后续数据都缓存好。

8 层请求接近结束时可能不满足这些条件。跨请求后 profile 改变时，也不能机械沿用同一个 C 和 K_s 。一次有限长突发的证明，不是“永远周期性 stall”的证明。

自然错开能成功的情况并不矛盾：当这个条件不成立、盘和链路有足够余量，且各卡存在可行的相位间隔时，一次等待可能把后续释放移到空闲段。**本条件不成立只表示证明未判定，不能反推一定能错开。**

7 客户端究竟要提供多少信息

7.1 三个信息等级，对应三种不同承诺

等级	可取得的信息	能做什么
1	各盘 Path0 总计数、目标分盘读量、当前剩余预算	对全未提交目标计算严格失败下界
2	每卡/请求/层的阶段计数、当前计算进度、已到达请求	重建一个可能的状态，做多层条件预测
3	真实 FIFO 归属顺序、服务剩余量、精确后续发射状态	在确定模型和已知输入下精确递推

现实客户端一般知道：已到达请求的 manifest、将来的层数与计算估计、本卡发了多少 I/O、收到了多少 HBM 完成回执、本卡自己的待接纳顺序。

但“SSD 已服务完、还没进入 HBM”与“仍在 SSD 队列里”不是同一阶段。如果系统只提供 HBM 完成回执，这两个数量不能直接分开。暖态验证中的每请求/层/SSU 的 SSD 完成计数，是额外的遥测能力，需要 SSD/SSU 或数据通路配合；不能称为只靠 Path 总队长就得到了全部状态。

同理，在途 SSD 的剩余量、精确 FIFO 顺序，不是普通客户端天然拥有的信息。新策略 A/B 不读取它们。独立预测器允许不提供，随后明确把结果标为条件预测。

7.2 NPUState 最容易填错的地方

NPUState 描述快照时刻的一张卡。函数把这一时刻当作 0，所有剩余时间都是相对于现在的 ms。

- request：当前请求的已知 manifest；没有请求时为 None。
- next_layer：下一层尚未开始计算的层号。正在算 L2 时应填 3，而不是 2。
- compute_end_in_ms：当前计算还剩多久；负数表示前一个 deadline 已经过去，卡正在等数据。
- queued_io_by_ssu：**第一待计算层**已提交、尚未离开各 SSD 的数量，包含在途 SSD I/O，不含 link-only I/O。
- unissued_by_ssu=None：这层读取还没激活；全 0 的元组表示已激活且全已提交，两者不同。
- link_pending_io：第一待计算层尚在接收队列中的 I/O 数，包含正在接收的一条。
- next_issue_in_ms、emitter_ssu_order：本卡客户端下一次可以发射的时间、尚未发完的 SSU 轮转顺序。它们不是 SSD FIFO 顺序。
- waiting_requests：已经到达、在本卡排队的后续请求。不要放入未来尚未到达的请求。

当前请求正在计算最后一层时，如果下一请求的 L0 已经被跨请求预取，这些 I/O 数量描述的是那个 L0；原请求仍放在 request，下一请求放在 waiting_requests 的第一位。

7.3 同样的总队长为什么会有不同结果

假设盘的 Path0 总共还剩 4 条 I/O，NPU0 和 NPU1 各占 2 条：

- 排列 A：NPU0、NPU0、NPU1、NPU1。
- 排列 B：NPU1、NPU1、NPU0、NPU0。

总队长、每卡数量都一样，NPU0 到齐时间却不同。NPU0 的计算因此不同步，后续层的发射时刻也跟着改变。

到了多盘，还会改变不同 SSD 的到达流如何在同一个 NPU 接收口合并。**信息不足不是公式复杂度的问题：不同真实状态映射成了同一个输入计数。**任何只接收这些计数的确定函数，都不可能同时精确还原这两种真实状态。

8 当前请求、新请求：最小可用的 Python 调用

8.1 用完整已知请求做冷启动闭环预测

```
from multi_ssu_stall_predictor import (
    Request, NPUState, forecast_baseline, predict_request_impact,
)
```

```

ssu_count = 6
placement = tuple(k % ssu_count for k in range(12))
states = [
    NPUState(Request(
        request_id=1000 + n,
        io_ssus_by_layer=(placement,),
        compute_ms=0.08 + (n % 4) * 0.04,
        layers=8,
    ))
    for n in range(32)
]

before = forecast_baseline(
    states, num_ssu=ssu_count, issue_order_seed=42,
)

```

这里 32 个请求都已经在时刻 0 到达、尚未开始读取；每层 12 条 I/O，每盘 2 条。io_ssus_by_layer 只有一个元组，表示所有层复用这份分盘布局。若不同层布局不同，就传入 8 个元组。

issue_order_seed=42 是复现实验冷启动时同刻发射排序的输入。在线接入一段已经运行的系统时，重新给 seed=42 并不等于知道真实随机数发生器已经走到哪里，不能据此声称暖态精确。

8.2 新请求进入后，分别检查自己和别人

```

new_request = Request(
    request_id=2000,
    io_ssus_by_layer=(tuple(k % ssu_count for k in range(48))),),
    compute_ms=0.2,
    layers=8,
)

impact = predict_request_impact(
    states, new_request, target_npu=0,
    num_ssu=ssu_count, issue_order_seed=42,
)

own_warm_stall_ms = sum(
    row["future_stall_ms"]
    for row in impact["candidate_layers"]
    if row["layer"] > 0
)

```

```
others_delay_ms = impact["existing_request_completion_delay_ms"]
print(own_warm_stall_ms, others_delay_ms)
print(impact["guaranteed"]) # False
```

这个新请求会排在 NPU0 的已有请求后面，不能越过它。比较方法是：

1. 在同一快照上预测没有新请求的后续时间线。
2. 只把新请求加入目标卡的已到达队尾，再预测一次。
3. 对相同 request_id、相同 layer 逐项相减。

输出有新请求自己的层等待，也有旧请求完成时间差。某张卡某层的差值可能是负数：新增工作会改变相位，局部层偶尔反而更早到齐。不能把所有负值截成 0 后，再声称那是完整因果差分。

上面是一个输入清楚、便于运行的冷启动例子。实际新请求在运行中到达时，应先按上一节采集 NPUSate 快照，再使用相同 API；不能把已发生的所有工作重新当成未开始的请求。

impact_order_sensitivity 默认测试正序/逆序、分组/轮转四种重建排列。它避免枚举 $32!$ 种顺序，但得到的最小值、最大值只是四个样本的范围，**不是所有真实队列顺序的保证上下界**。

9 哪些等待有机会优化，哪些不能靠 QoS 消除

9.1 先算每盘和每卡的长期需求

对一张持续有任务的卡，如果请求会变化，理想无 stall 的长期需求应按工作与计算之比计算：

$$d_{i,s} = \frac{\sum_r V_{i,r,s}}{\sum_r C_{i,r}^{total}/1000} \text{ GiB/s.}$$

这里分子是这些请求的全部层在盘 s 上的读取 GiB，分母是这些请求的全部层计算秒数。不能把每请求 V/C 简单做算术平均后，称为长期平均带宽；短计算请求和长计算请求占用的时间不同。

理想长期没有资源超载，至少需要：

$$\sum_i d_{i,s} \leq B_s \text{ 对每个盘分别成立,}$$

$$\sum_s d_{i,s} \leq L_i \text{ 对每张卡分别成立.}$$

只检查 $\sum_{i,s} d_{i,s} \leq \sum_s B_s$ 太弱，遗漏了热点盘和 NPU 接收限制。有限请求队列的这个比值是工作负载指标，不是“真实每个时间段都有这么多流量”的事实；有突发到达时还需检查更短的 deadline 窗口。

9.2 单层独占仍来不及，不该怪其他 NPU

忽略全部排队和命令启动，单层完成也至少需要：

$$T_{solo}^{fluid} = 1000 \max \left(\max_s \frac{V_{i,s}}{B_s}, \frac{\sum_s V_{i,s}}{L_i} \right).$$

若当前剩余预算连这个理想时间都小，改变 Path 或提高 CIR 也不能让它按原 deadline 完成。实际命令粒度和启动尾延迟只会使严格要求更紧。

同理，在任意一个必须完成的时间窗内，如果某盘的必需工作量超过该盘能服务的量，或者某卡的必需接收量超过链路能力，至少有一个 deadline 会失守。动态 CIR 只能选择谁先完成，不能创造带宽。

9.3 容量检查通过，也不等于无 stall

上述是必要条件，不是充分条件。还可能存在：

- Path0 的长批次把短预算任务挡在后面。
- 读取激活得太晚，整体吞吐够但 deadline 前的时间不够。
- 多盘组成的同一层有一个慢分量；其他盘的提前完成不能抵消它。
- 同刻完成造成 NPU 接收突发。
- 后续请求 profile 变化，把原先可行的错开相位打破。

这些是调度有机会改善的部分，但改善多少仍需要同输入实验。

10 策略 A：分盘、分卡联合规划 CIR

10.1 不只是给每个 SSU 各做一次独立 EDF

策略 A 给每个 NPU 在每个 SSU 上一个专用 Path，32 卡使用每盘可用 256 条 Path 中的 32 条。原生映射为：

$$path(i) = (i \bmod 8) \times 32 + \lfloor i/8 \rfloor.$$

所有盘上“这层剩余读量的向量”在代码中叫一个 PendingCoflow。它只代表某张卡当前已经激活、尚未到齐的一层，不是把所有未来层立即入队。

控制器输出速率矩阵 $r_{i,s}$ ，同时约束：

$$\sum_i r_{i,s} \leq B_s, \quad \sum_s r_{i,s} \leq L_i.$$

前者保护每盘容量，后者避免**规划值**要求一张卡接收超过 50 GiB/s。实际 SSD 仍按原来的命令级仲裁器工作。

10.2 已实现的可比较规则

- demand：根据剩余读量与计算时间构造需求，用联合 max-min 分配作为对照。
- deadline：按当前数据 deadline 排序，再为某层各盘分量同步分配。
- least_slack：改按 deadline 减去独占理想读取时间排序。
- deadline_reserve：先尝试给每层保留 $W/(D - t)$ ，再分配剩余带宽。

同步分配使用一个共同进度标量 θ_i ：

$$r_{i,s} = \theta_i V_{i,s}.$$

这样某层的容易读取部分不会在规划中远远跑在瓶颈盘之前。代价是它可能让某些暂时无关的盘没有满额分配，所以不能预先断言这总比各盘独立贪心更好。

10.3 CIR 不是实际瞬时速率，也不是 PIR

这一点是本实验最重要的实现边界之一：**CIR 表里满足行上限，不代表实际流量绝不会把 HBM 接收口打满**。原生仲裁器保留历史虚拟完成标签；正在服务的命令不能抢占；CIR=0 的 Path 仍可能借用空闲带宽。

因此，“CIR 偏向最早 deadline”不是精确 EDF，也不是硬限速器，更不是无 stall 保证。当前 sim.py 明确拒绝有限 PIR，实验使用的是不限 PIR。虽然策略权限允许设置 PIR，本轮并未凭空补出一个未经验证的有限 PIR 模型。

在线控制使用已到达 manifest、真实计算进度、HBM 完成通知，不读取 SSD FIFO。只有 HBM 回执时，已经离开 SSD 但尚未进入 HBM 的字节还会保守计入剩余工作，这会产生一定过度预留。

控制在层开始、整层到齐、可选的某盘分量到齐以及原生请求边界触发。可使用最小更新间隔合并通知。实验会记录控制 CPU 时间、观测次数和寄存器写入次数；**这些 CPU 时间和通信延迟尚未真实推迟数据面事件**，所以不能把模拟收益直接当成硬件净收益。

11 策略 B：新请求到达时再选 NPU

策略 B 保留 A 的 Path/CIR 能力，只额外允许在实际到达时选一张 NPU。已在执行的请求不迁移；到达时间不改变；数据仍在原 manifest 指定的 SSD 上，**选 NPU 不等于重新放置 SSD 数据**。

multi_ssu_npu_assignment.py 的纯函数输入每张卡已经到达请求的剩余计算 C_i 、剩余分盘工作 $V_{i,s}$ ，以及新请求的总工作向量。它对每个候选 NPU 加入新工作后计算：

$$\rho_{disk} = \max_s \frac{\sum_i 1000 V_{i,s} / C_i}{B_s},$$

$$\rho_{link} = \max_i \frac{1000 \sum_s V_{i,s} / C_i}{L_i},$$

$$score = t + \max_i C_i \max(1, \rho_{disk}, \rho_{link}).$$

选择 score 较小的候选。这是一个**整个已知 backlog 的近似完成时间目标**，不是新请求单独完成时间的精确解；它没有还原 FIFO 顺序或逐层 deadline。

空卡且没有工作时，其需求项按 0 处理。不同 SSU 分布即使总 I/O 数相同，也能得到不同的得分和选卡结果；这避免了把 6 盘简单当成一个 240 GiB/s 的盘。

同时保留 compute 消融：只按剩余计算量分配。它有助于判断改进究竟来自负载均衡还是分盘 I/O 感知。I/O 感知启发式并不保证每个 trace 都优于这个简单对照。

分配器只读取已到达请求；更新仿真器未来到达计数是维护记账，不会把这个计数用于选择，也不读取未来请求内容。数据和到达时刻不变的约束由集成测试检查。

12 怎样判断“数学和 Python 匹配仿真”

12.1 需要分别报告冷态和暖态

运行：

```
python -m unittest test_multi_ssu_stall_predictor
python -m unittest test_multi_ssu_qos_controller
python -m unittest test_multi_ssu_npu_assignment
python validate_multi_ssu_stall_predictor.py
```

验证输出保存于：

```
data/validation_multi_ssu_predictor.json
```

冷启动对照从同一组已知请求、空队列、明确的提交 seed 开始。独立 Python 自己计算全部 SSD、接收、计算、再预取事件，不读取仿真器未来 trace。如果逐层 io_ready_ms、compute_start_ms、future_stall_ms 一致，证明的是**这个确定模型和这些输入上的算法对齐**。

暖态对照故意不读取真实 SSD FIFO、不读取原生随机数发生器当前位置。它从计数重建队列，必须分别报告时间误差、stall 漏报和误报。即使某次总请求完成时间很接近，也不能掩盖中间层的漏报。严格下界另外验证；不要把条件预测的误差当成严格界限失效。

12.2 本轮已经验证到了什么范围

独立验证覆盖 6、7 SSU 的变请求、热点布局、接收链路瓶颈，并包含直接取自 data 的不同长度和 NQL。10 组冷启动案例共 3088 个层，读取到齐、计算开始、未来 stall 三项逐层误差均为 0。这里的“0”是这些确定性用例与原仿真器一致，不是统计上证明所有输入都正确。

暖态验证在三个不同运行时刻加入一个新请求。最大读取到齐时间误差及非 L0 stall 的分类错误如下：

SSU 数	快照时刻 ms	最大到齐误差 ms	漏报 / 误报
6	0.002	0.303724	0 / 0
6	0.040	0.535275	1 / 0
6	2.000	1.020215	0 / 0
7	0.002	0.233628	0 / 0
7	0.040	0.462701	1 / 0
7	2.000	1.261261	0 / 1

2 ms 快照时，6 盘案例有 24 张卡正在计算、接收口合计有 108 条 I/O；7 盘案例有 15 张卡正在计算、接收口合计有 50 条 I/O。因此暖态检查不只挑选了刚启动、没有计算重叠的简单状态。

另外，512 个全未提交层和 192 个部分提交层，共 704 次严格下界对照，没有发现下界超过实际到齐时间。这支持下界实现与上述证明一致；它不意味着下界筛查能发现全部 stall，也不意味着暖态重建队列的误差已经消失。

实际结论是：冷启动模型已精确对齐；仅给暖态计数时仍有误差和漏报，不能把这版条件预测直接用作“保证不会影响任何请求”的准入承诺。

本轮独立预测器描述的是 Baseline Path0。它没有实现全部 QoS 虚拟标签仲裁，**不能直接用其 FIFO 输出声称预测了 A/B 的精确逐层完成时间**。A/B 的真实效果仍由未修改核心的项目仿真器验证。

12.3 策略比较还要防止三个偏差

第一，同一个拓扑下，Baseline、Once/layer、A、B 应使用完全相同的请求、到达时刻、SSD placement 和提交 seed。B 只在到达后修改实际分配。跨 6 盘与 7 盘比较时，要分清“相同用户输入换硬件”与“分别构造接近各自容量的输入”；后者的 profile 配比可能不同，不能当成纯硬件对比。

第二，平均利用率使用共同的固定窗口，例如 $[1000, 2000]$ ms：

$$U = \frac{\sum_{i,l} |[S_{i,l}, E_{i,l}] \cap [1000, 2000]|}{32 \times 1000}.$$

要同时确认所有卡在整窗都有活跃请求。没有任务可做的 idle 不能误算为 I/O stall；每个策略自行移动窗口，也可能制造相位选择偏差。

第三，再报告完整相同请求集的完成时间、到达至完成延迟和调度成本。仅窗口内平均利用率提高，可能同时有某些请求长时间排在客户端，或者把更重的工作推到了窗口之外。

data 给出 profile 参数，不提供真实用户请求频率和真实到达轨迹。本实验可以使用真实目录中的长度、NQL、计算时间，但人工配比和到达规则仍是**构造的工作负载**，不能叫作生产环境实测分布。

13 最后整理成实际使用顺序

1. 先确认完整 176 KiB 单位、每层计算时间、分盘 placement 和每卡接收上限。
2. 对全未提交的目标层，用 `screen_current_layer` 排除那些最理想也赶不上的情况。
3. 新请求到达时，读取已到达状态，用 `predict_request_impact` 比较加入前后；明确不知道 FIFO 时只能作条件预测。
4. A/B 用便宜、可获取的计数和 deadline 在线决策；不要每次到达都把全 I/O 多层展开成本当成免费。
5. 用原仿真器的同输入对照验证收益、误差和失败案例，再判断是否值得增加真实遥测和控制开销。

核心判断是：**共享的每个瓶颈资源都要单独计算，读取还必须赶上具体层的 deadline。总平均带宽够，只是必要检查的一部分。**

本说明对应的项目事实来自 `sim.py` 的命令级 SSD 调度与 PIR 限制、`continuous_batch_sim.py` 的预取及每卡 FCFS 接收实现、上述三个新增 Python 文件及其测试。实际 SSD 固件的内部并行度、服务波动、读写竞争、控制链路时延和模型计算时间误差，不包含在这些确定性公式里。